Observability Day
NORTH AMERICA

Auto Deriving Threat Signals from OTel CPU Profiles & OBI Network Flows

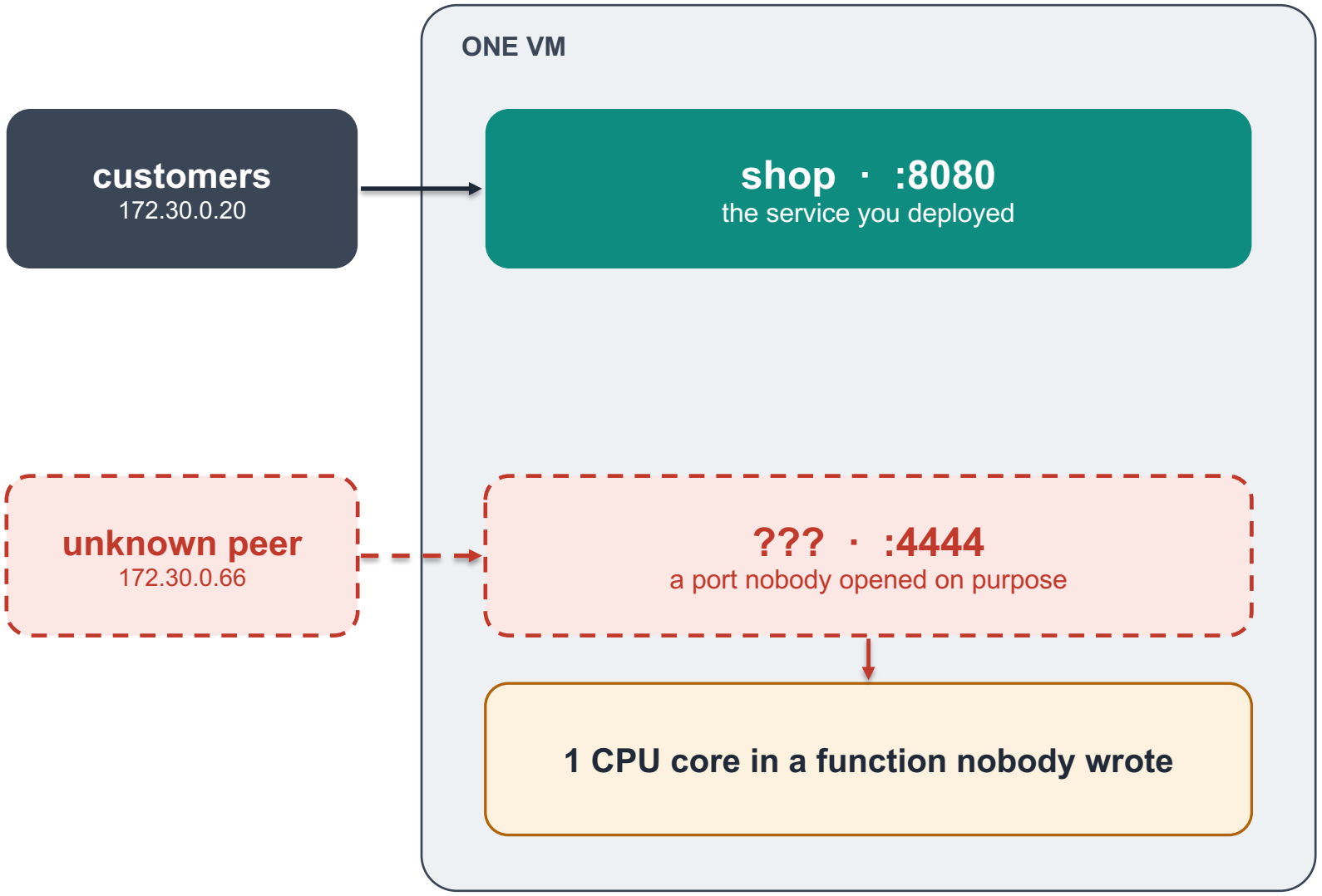
Observability Day · KubeCon + CloudNativeCon North America 2026

Shivanshu Raj Shrivastava · Engineer, Independent

Ekansh Gupta · Software Engineer, Bindplane (Dynatrace)

No detection rules. Two signals you already collect. One PID.

One VM, two problems



1 Unknown traffic

- Which port?
- Which peer?
- Which process answers?

2 Unknown code

- What is that process running?
- Did we ship it?

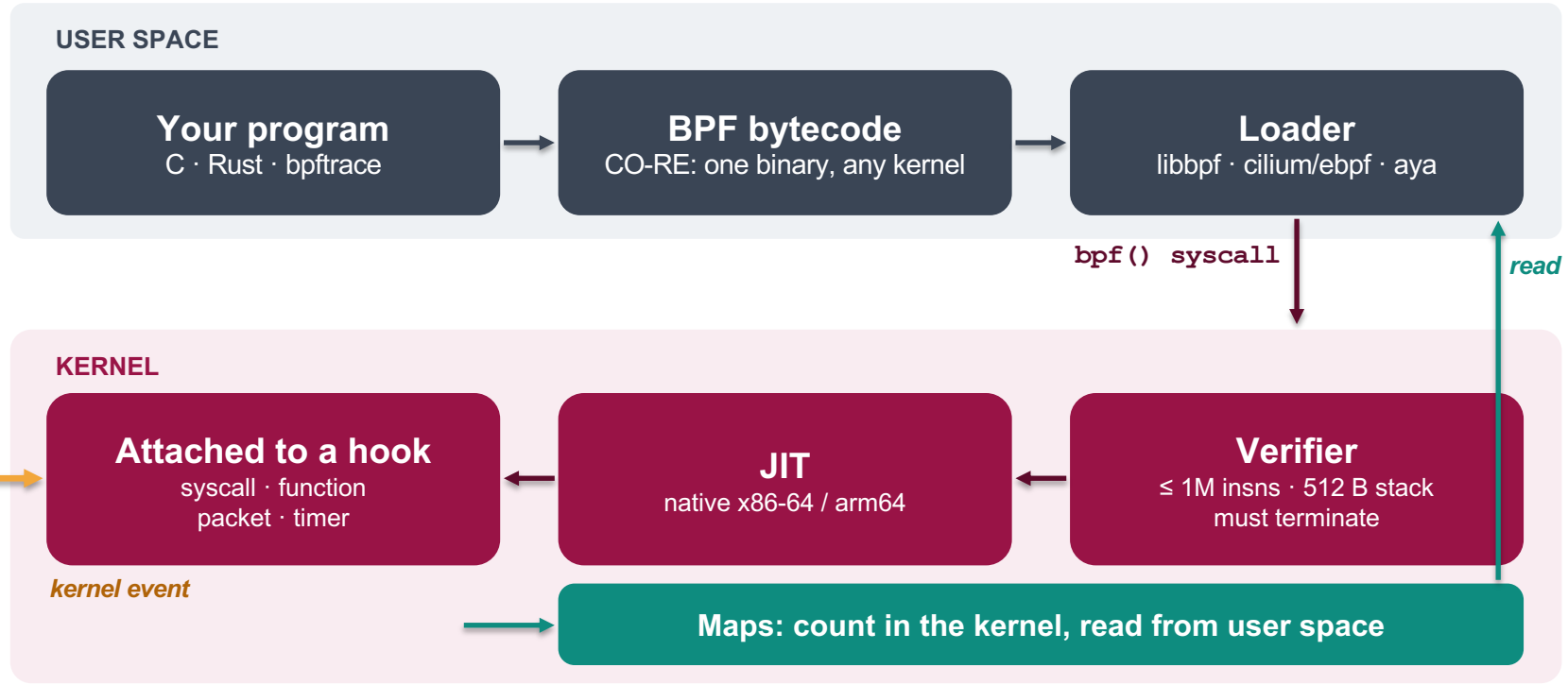
The idea: two signals meet on one PID



1. Both come from eBPF: nothing installed in the app, no code change.
2. Both report the host PID. That is the join.
3. No rule mentions mining or port 4444. The only logic: this is new.

eBPF in one picture

Observability Day
NORTH AMERICA



2014

Linux 3.18 ships the bpf() syscall

+15 ns

one kernel tracepoint hit

0

app changes, restarts or kernel modules

1992	2014	2016	2018	2020	2021	2023	2024	2025	Oct 2026
BPF packet filters, tcpdump	eBPF in Linux 3.18	XDP	BTF, CO-RE	BPF LSM + ring buffer	eBPF Foundation	Cilium graduates	Falco graduates	Beyla donated → OBI	OBI 1.0 · Profiles alpha

ebpf.io/what-is-ebpf · kernel bpf_design_QA · Cloudflare ebpf_exporter benchmark, Linux 6.5-rc1 · opentelemetry.io/blog/2026/obi-v1-release

eBPF tools: many names, same four moves

Observability Day
NORTH AMERICA

Networking	Observability	Security	Toolkits
Cilium	OBI *demo*	Falco	bpftrace
Hubble	OTel eBPF profiler *demo*	Tetragon	bcc
Calico eBPF	Pixie	KubeArmor	libbpf · cilium/ebpf · aya
Katran	Parca · Pyroscope	Tracee	Inspektor Gadget
Retina	Coroot · DeepFlow		bpfman

Every one of them makes the same four moves:

1 · Attach
a verified program to a hook

→

2 · Filter
in the kernel, before it costs anything

→

3 · Aggregate
in a map: bytes, counts, stacks

→

4 · Ship
summaries to user space, not events

The difference: which hook, and what happens after step 4.

Versions and CNCF levels as of Nov 2026 · OBI and the OTel eBPF profiler are the two signals in today's demo

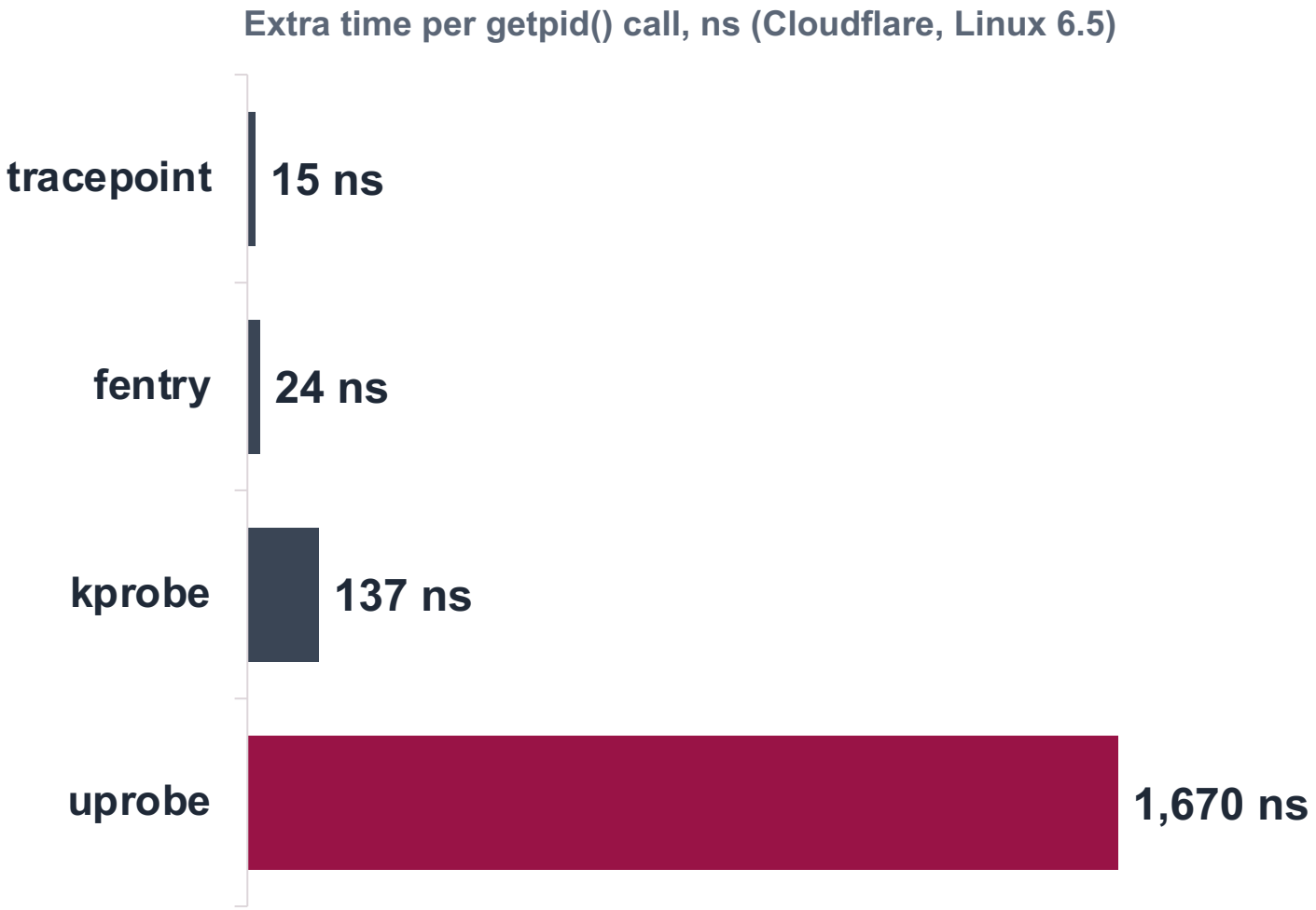
Where you can hook: from process to NIC

Observability Day
NORTH AMERICA

Your process	uprobe	USDT	perf_event ★	function calls, stacks → profiles
Syscalls	tracepoints	raw_tp · tp_btf		every syscall (Falco lives here)
Kernel functions	kprobe	fentry · fexit		tcp_connect, tcp_sendmsg…
Security hooks	BPF LSM			can say no (Tetragon, KubeArmor)
Sockets	sock_ops	cgroup connect	sk_msg	connect, listen, round trips
Network stack	TC	socket filter ★		5-tuple + bytes → flows
Driver / NIC	XDP			raw frames, DDoS drop

★ the two hooks today's demo uses: perf_event (CPU profiles) and the socket filter (network flows)

Four probes, four prices



100×

a user probe costs about a hundred kernel probes: every hit traps into the kernel

< 1%

CPU for a whole-node profiler: it runs on a timer, not on your code path

Hook in the kernel when you can, in user space when you must.

Cloudflare ebpf_exporter benchmark (Linux 6.5-rc1, getpid baseline 117 ns) · opentelemetry-ebpf-profiler README (1% CPU / 250 MB ceiling)

What each hook hands a defender

Observability Day
NORTH AMERICA

XDP

sees every DNS answer

signal: **a domain nobody here ever resolved**

TC · socket filter

sees every flow's 5-tuple and bytes

signal: **pod → pod on a port nobody used: lateral movement**

sock_ops

sees connect, listen, round trips

signal: **a brand-new listening port · beaconing on a timer**

kprobe tcp_connect

sees every outbound connect

signal: **first-ever egress · port scans: SYNs, no data**

Count in the kernel, ship summaries: OBI sends one record per 5-tuple every 5 seconds, not one per packet.

docs.ebpf.io BPF_PROG_TYPE_SOCK_OPS · bcc tcpconnect / tcplife · OBI bpf/ sources at v1.0.0

Same kernel, four bets

Observability Day
NORTH AMERICA

Falco	Tetragon	Calico · Cilium	OpenTelemetry
HOOKS	**HOOKS**	**HOOKS**	**HOOKS**
syscall tracepoints	kprobe · LSM · uprobe	TC · XDP · sockets	uprobe · kprobe · socket · perf
YOU WRITE	**YOU WRITE**	**YOU WRITE**	**YOU WRITE**
rules: 25 ship, you add yours	TracingPolicy CRDs	network policy	**nothing**
YOU GET	**YOU GET**	**YOU GET**	**YOU GET**
alerts	events · SIGKILL	allow / drop · Hubble flows	flows · spans · profiles, OTLP



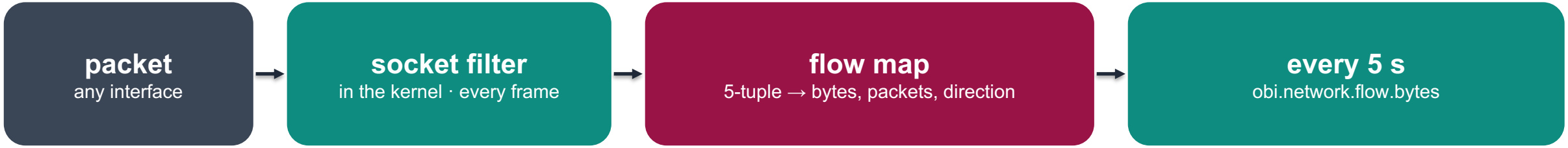
you write the detection — **the data describes the workload**

Hubble's flows and OBI's flows come from the same kind of hook. The security picture is already in the observability data.

falco.org/docs · falcosecurity/rules · docs.tigera.io · docs.cilium.io · tetragon.io · opentelemetry.io/docs/zero-code/obi

Signal 1: a network flow (OBI)

Observability Day
NORTH AMERICA




```
obi.network.flow.bytes            723 B/s
  src.address    172.30.0.66     # no CIDR name
  dst.address    172.30.0.10     # dst.cidr = shop
  server.port    4444
  direction      request
```

Turn these on

- addresses and ports are off by default (cardinality)
- name your CIDRs: identity without Kubernetes
- socket filter coexists with Cilium's TC

Who talked to whom, on which port, how much. Nobody wrote a rule to get this.

Signal 2: a CPU profile (eBPF profiler)

Observability Day
NORTH AMERICA




```
profile   pid=37194   exe=python3.12   0.76 cores
  100.0%  grind_nonce
          mine_forever
          Thread.run
          Thread._bootstrap
```

Why it's cheap

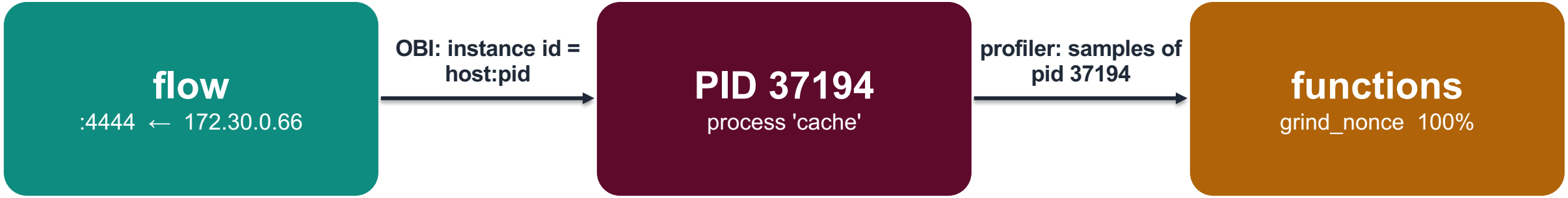
- on a timer, not on your code path
- under 1% CPU and 250 MB: the project's own ceiling
- OTel Profiles: alpha since March 2026

What code is on the CPU, per process, all the time. Nobody instrumented anything.

github.com/open-telemetry/opentelemetry-ebpf-profiler · sample captured on the demo rig

The join: port → PID → functions

Observability Day
NORTH AMERICA



On a VM

The key is the host PID. OBI and the profiler both report it, so they agree without any extra lookup.

On Kubernetes

cgroup → container.id → pod → owner. Same join, one level up, via the Collector's k8sattributes.

Where it breaks

- hostNetwork pods resolve to the node · NAT can hide pod IPs, depending on which interface you hooked
- name services the same way in both tools, or the join fails quietly

OBI pkg/internal/traces/read_decorator.go (instance id = host:pid) · opentelemetry-ebpf-profiler reporter (process.pid, container.id)

Bonus: logs carry the trace id, no SDK

Observability Day
NORTH AMERICA

request arrives
OBI opens a span

→

outgoing call
sk_msg injects traceparent

→

app writes a log
write() buffer gets trace_id

→

log shipper
Collector: TraceId on the record

```
{"level":"warn","msg":"spawned /bin/sh for request",
 "trace_id":"4bf92f3577b34da6a3ce929d0e0e4736",
 "span_id":"00f067aa0ba902b7"}
```

Why a defender cares: the log line that says a shell was spawned now points at the request that caused it, and that request has a pod and a flow.

Fine print

- off by default
- kernel 6.0+ for the log rewrite
- Go needs CAP_SYS_ADMIN
- buffered stdout is invisible

opentelemetry.io/docs/zero-code/obi/trace-log-correlation · OBI bpf/tpinjector (sk_msg)

Baselines, not rules

Observability Day
NORTH AMERICA

LEARN
5 calm minutes:
ports, peers, hot functions

→

WATCH
every minute: new port?
new peer? new hot code?

→

EXPLAIN
follow the PID:
port → process → functions

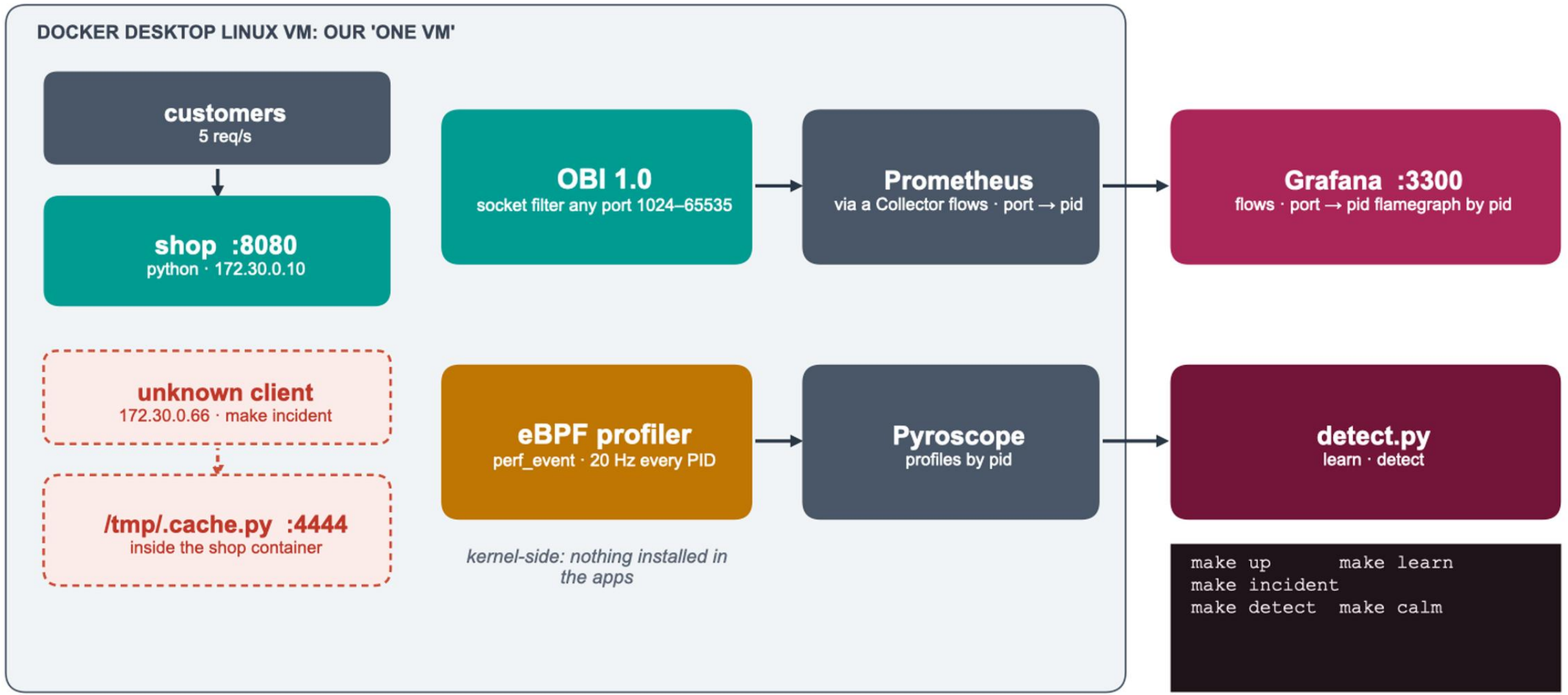
```
# detect.py: the whole detection logic
for (port, peer), bps in inbound().items():   # OBI flows
    if port in base['ports'] and peer in base['peers']:
        continue                                  # seen it
    svc, pid = owners()[port]    # OBI: host:pid
    cores, funcs = profile(pid) # profiler, by pid
    report(port, peer, svc, pid, cores, funcs)
```

What it is not

- no signatures
- no ML model
- no rule names a port, a binary or a function
- one stdlib Python file

The demo rig: one laptop, Wi-Fi off

Observability Day
NORTH AMERICA



otel/ebpf-instrument:v1.0.0 · otel/opentelemetry-collector-ebpf-profiler:0.164.0 · Prometheus · Pyroscope · Grafana · github.com/eKuG/obs-day-na26

Demo: catch it without a rule

1 **Calm** — 30 s

`make detect` → nothing new: one port, one peer

2 **Incident** — ~30 s to show

`make incident` → a listener on :4444 + a busy loop

3 **Follow the PID** — 90 s

Grafana: new port → pid → flamegraph

4 **Auto-derive** — 30 s

`make detect` → port, peer, pid, hot function

Grafana localhost:3300 · Prometheus :9090 · Pyroscope :4040 · backup frames at the end of the deck

What it catches, what it misses

Observability Day
NORTH AMERICA

Catches

- listeners nobody planned
- peers nobody knows · first-ever egress
- hot code nobody shipped: miners, scanners, exfil loops
- the same hot code hiding inside a known process

Misses (today)

- low and slow: below 20 Hz, or tiny flows
- port → PID here uses OBI's request metrics: the listener must speak a protocol OBI parses
- payloads: it says who and what, not why
- a box already owned when you learned the baseline

Take it home: three steps

Observability Day
NORTH AMERICA

1 Flows: addresses + ports

```
network:
 source: socket_filter
 cidrs:
   - {cidr: 10.0.0.10/32,
      name: shop}
attributes:
 select:
   obi_network_flow_bytes:
     include: [src.address,
       dst.address, server.port]
```

2 Profiles that keep the PID

```
receivers:
 profiling:
   samples_per_second: 20
# service.name = <exe>:<pid>
set(resource.attributes
 ["service.name"],
 Concat([exe, ":", pid],
 ""))
```

3 Baseline, then follow the PID

```
learn:   ports, peers,
        hot functions
detect:  anything new?
        port -> pid
             -> functions
# no rule names a port,
# a binary or a function
```

Everything in the demo: github.com/eKuG/obs-day-na26

`cd rig` · `make up` · `make learn` · `make incident` · `make detect`

opentelemetry.io/docs/zero-code/obi · github.com/open-telemetry/opentelemetry-ebpf-profiler



Thank you · Questions? · github.com/eKuG/obs-day-na26

Backup: the demo, frame by frame

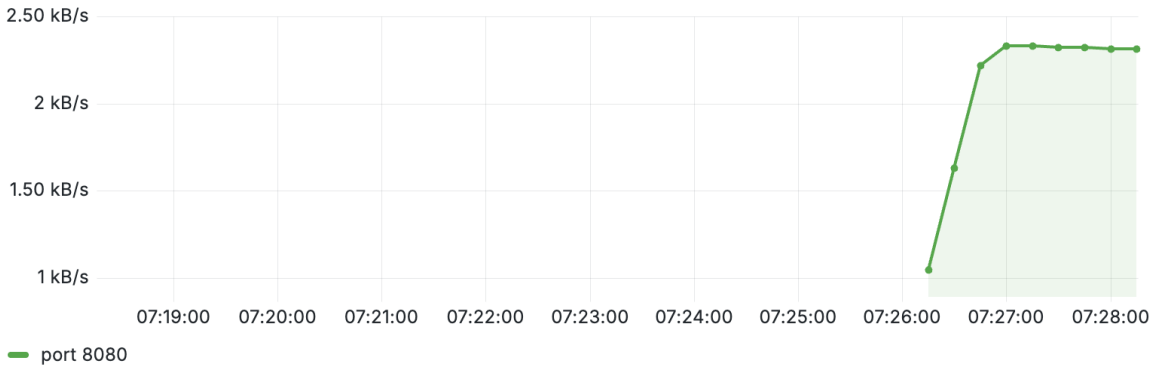
Captured on this rig before the talk. Shown only if the live run misbehaves.

Backup · calm: one port, one peer

Observability Day NORTH AMERICA

Port → PID (OBI) 8080 | Last 10 minutes | Refresh 10s

1 · Bytes/s into the shop, per port (OBI network flows)



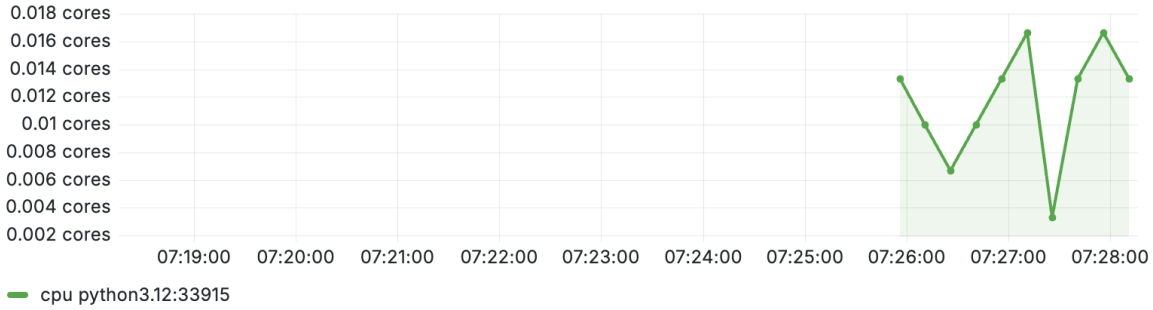
2 · Who talks to the shop, on which port (OBI network flows)

port ↓	peer	peer cidr	bytes/s
8080	172.30.0.20	customers	2314

3 · Which process answers on each port (OBI: service.instance.id = host:pid)

port	process	host:pid	req/s
8080	python3.12	docker-desktop:33915	4.6

4 · CPU per Python process (eBPF profiler, 20 Hz)



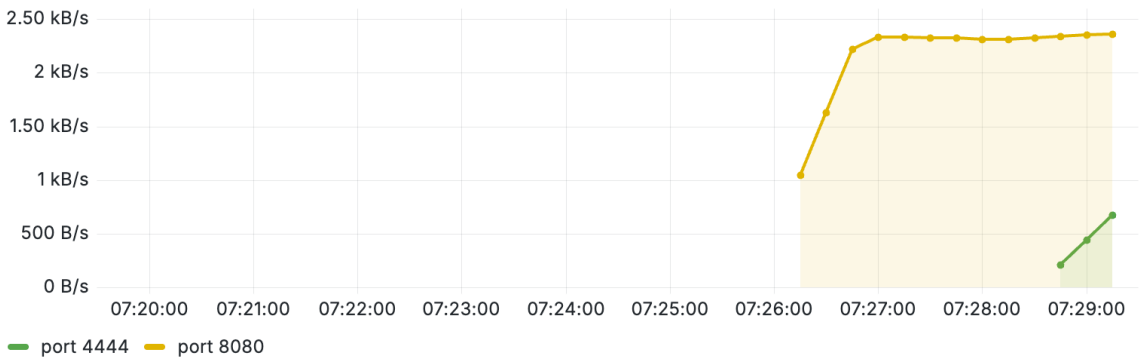
Before the incident: only port 8080, only the customers, only python answering. make detect: nothing new.

Backup · incident: port, peer, process

Observability Day NORTH AMERICA

Port → PID (OBI) 4444 | Last 10 minutes | Refresh 10s

1 · Bytes/s into the shop, per port (OBI network flows)



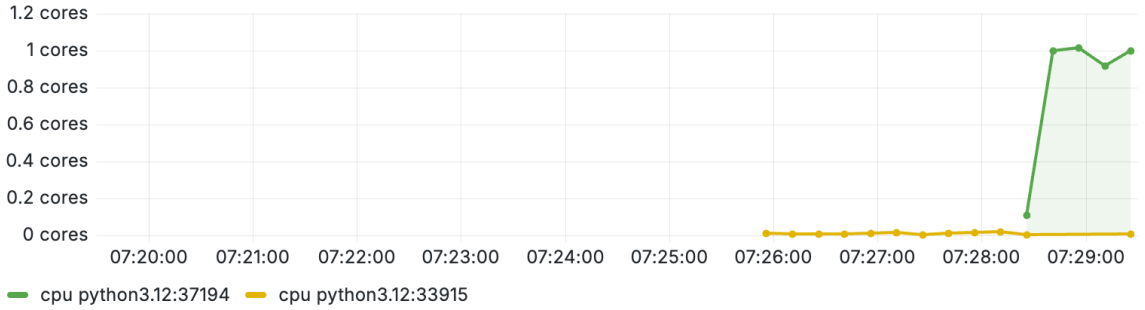
2 · Who talks to the shop, on which port (OBI network flows)

port ↓	peer	peer cidr	bytes/s
8080	172.30.0.20	customers	2372
4444	172.30.0.66	vm-bridge	898

3 · Which process answers on each port (OBI: service.instance.id = host:pid)

port	process	host:pid	req/s
8080	python3.12	docker-desktop:33915	5.0
4444	cache	docker-desktop:37194	1.8

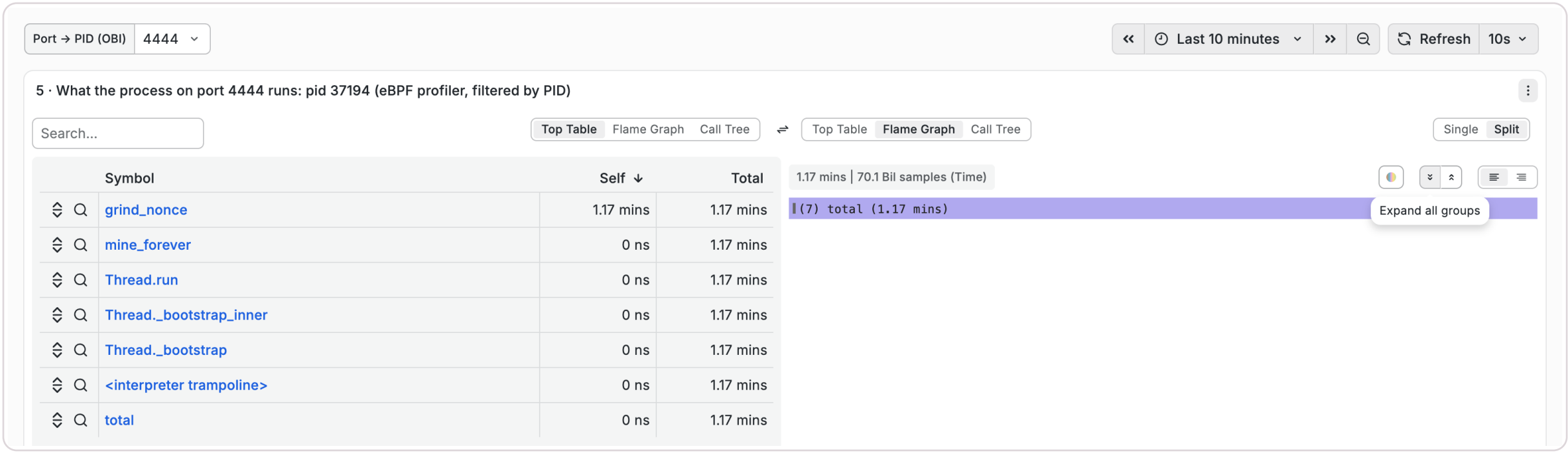
4 · CPU per Python process (eBPF profiler, 20 Hz)



Port 4444 appears, from 172.30.0.66 (no CIDR name). OBI: port 4444 is answered by 'cache', pid 37194. Its CPU jumps to ~1 core.

Backup · follow the PID: the profile

Observability Day
NORTH AMERICA



Profiler samples filtered by pid 37194: 100% of its CPU is self time in grind_nonce (Top Table, left).

Backup · detect.py output

Observability Day
NORTH AMERICA

```
$ make detect           # calm
nothing new on shop: every port, peer and hot function matches the baseline

$ make incident; sleep 30; make detect

[NEW PORT + NEW PEER] shop:4444  <-  172.30.0.66   705 B/s   (OBI network flow)
   served by 'cache', host pid 37194   (OBI process discovery: service.instance.id)
   pid 37194 uses 0.76 CPU cores   (eBPF profiler, last 60s)
   100.0%  grind_nonce   <- never seen in this workload
```